Gymnasium Oberwil

Maturarbeit

Bildsynthese von menschlichen Gesichtern mithilfe von Kuenstlicher Intelligenz

Eine Anwendung eines Autoencoders

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019
LaTeX

Inhaltsverzeichnis

1 Vorwort **3**

2 Einleitung **4**

3 Konventionen und Notation **5**

4 Machinelles Lernen - Theorie **6**
4.1 Allgemeine Begriffe . . . 6
4.1.1 Daten . . . 6
4.1.2 Modelle . . . 6
4.2 Training . . . 7
4.2.1 Kostenfunktionen . . . 7
4.2.2 Gradientenverfahren . . . 8
4.2.3 Stochastisches Gradientenverfahren fuer Machine Learning . . . 9
4.2.4 Adam Optimizer . . . 10
4.3 Künstliche Neuronale Netze und Deep Learning . . . 11
4.3.1 Perzeptron . . . 11
4.3.2 Biologische Analogie . . . 12
4.3.3 Erweiterung der Kuenstlichen Neuronen . . . 12
4.3.4 Topologie der Kuenstlichen Neuronalen Netzen . . . 14
4.3.5 Vorwaertspropagierung . . . 15
4.3.6 Gewichte und Neigungen initalisieren* . . . 17
4.3.7 Rueckwaertspropagierung . . . 17
4.3.8 Universal Approximation Theorem . . . 18
4.4 Convolutional Neural Networks . . . 19
4.4.1 Bilder als 3D-Matrizen . . . 19
4.4.2 Filter in der Bildverarbeitung . . . 20
4.4.3 Filter als Faltungen (Convolutions) . . . 20
4.4.4 Filter fuer farbige Bilder . . . 21
4.4.5 Filter in CNNs . . . 21
4.4.6 Convolutional Layer . . . 22
4.4.7 Pooling Layer . . . 22
4.4.8 Fully-connected Layer . . . 22
4.4.9 Das Netzwerk und sein Aufbau . . . 22
4.4.10 Eigenschaften von CNNs . . . 23
4.5 Autoencoder . . . 24
4.5.1 Autoencoder . . . 24
4.5.2 Bedeutung von Autoencodern . . . 24
4.5.3 Variational Autoencoder? . . . 24
4.5.4 Detangled Autoencoder? . . . 24
4.6 Programmiertools . . . 24
4.6.1 Tensorflow . . . 24

5 Entwicklung eines Autoencoders **25**
5.1 Tools . . . 25
5.1.1 Python . . . 25
5.1.2 Tensorflow . . . 25
5.1.3 Keras . . . 25

5.2 Idee 25
5.3 Voraussetzungen 25
5.3.1 Trainingsdaten 25
5.4 Programmieren 25

6 Diskussion? 26

7 Schlusswort 27

1 Vorwort

2 Einleitung

Die Leitfrage lautet: **Wie können mithilfe von Machine Learning Bilder von künstlichen meschlichen Gesichtern generiert werden?**

3 Konventionen und Notation

Beschriftung

x	ein Skalar (Zahl)
$\mathbf{v}$	ein Vektor
v_i	i-te Vektorkomponente
$\mathbf{M}$	eine Matrix
$\mathbf{M}_{n\times m}$	Matrix mit n Zeilen und m Spalten
$(\mathbf{M})_{i,j} =$	Matrixelement in der i-ten Zeile und der j-ten Spalte
$m_{i,j}$	”
$\mathbf{M}^{\mathrm{T}}$	eine transponierte Matrix
$\overline{\mathbf{T}}$	ein Tensor
$\mathbb{B}$	eine Menge
$\mathcal{A}, \mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}, \cdots$	ein Funktionenraum
$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \cdots$	?
$\mathfrak{a}, \mathfrak{b}, \mathfrak{c}, \mathfrak{d}, \mathfrak{e}, \mathfrak{f}, \mathfrak{g}, \cdots$	?

Konventionen

- Falls ein (Zeilen- oder Spalten-) Vektor $\mathbf{v}$ als Argument in eine Funktion f gegeben wird, bedeutet dies, dass die einzelnen Komponenten des Vektors die Parameter der Funktion sind.
$$f(\mathbf{v}) = f(v_1, \ldots, v_n)$$

- Falls die Klammern, in denen der Vektor $\mathbf{v}$ steht, als eckige Klammern [] geschrieben sind, bedeutet dies, dass es sich um die Vektorisierung einer Funktion f handelt, somit wird die Funktion auf jede Vektorkomponente angewandt.
$$f[\mathbf{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

ERWEITERN

4 Machinelles Lernen - Theorie

Der Themenbereich des **Machinellen Lernens** beschäftigt sich mit Algorithmen und mathematischen Modellen, welche von selber lernen, Probleme zu loesen. Hierzu werden keine expliziten Instruktionen fest einprogrammiert (engl.: hard coding), sondern das Modell wird trainiert und optimiert sich von selbst. Dabei wird das Modell mit Trainingsdaten befuettert, mithilfe eine Korrelation zwischen den Inputdaten und den Outputdaten erlernt werden soll. Der Zusammenhang der Daten wird von den Modellen verallgemeinert und das Modell kann dann Vorhersagen fuer neue Daten machen.

4.1 Allgemeine Begriffe

4.1.1 Daten

Der **Trainingsdatensatz** $\mathbb{T}$ beinhaltet gewisse **Attribute**, die Merkmalstypen der Inputdaten. Die **Features** sind dann die spezifischen Werte der Attribute eines Trainingssample. Die sogenannten **Labels** sind die erwarteten Outputs, welche der Algorithmus vorherzusagen hat. Diese Vorhersagen werden mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (keine Ueberanpassung) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs anhand von Mustern und Gesetzmassigkeiten.

Um eine endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz, welcher nicht Teil des Trainingsdatensatz ist, genutzt um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

Die Inputs/Features werden in einem Vektor $\mathbf{x} = (x_1, x_2, \ldots, x_n)^{\mathrm{T}}$ und die Outputs/Vohersagen in einem Vektor $\mathbf{y} = (y_1, y_2, \ldots, y_m)^{\mathrm{T}}$ zusammengefasst. Das gleiche macht man mit den Labels, wobei diese mit $\hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m)^{\mathrm{T}}$ bezeichnet werden.

BESSERS BEISPIEL -> UM SPAETER KONKRETES MODELL ZU ERKLAEREN Am Beispiel eines Datensatz, welcher fuer Bilderkennung von Hunden und Katzen genutzt wird, werden diese Begriffe verdeutlicht. Die Inputdaten sind die Fotos von Hunden und Katzen. Deren Attribute sind die moeglichen RGB-Farbenwerte (drei Zahlen, welche von 0 bis 255 reichen) aller Pixel. Die Features sind die spezifischen RGB-Werte der Pixel fuer die einzelnen Bilder. Die Outputdaten sind die Labels, welche in diesem Fall angeben, ob ein Hund oder eine Katze auf dem Bild gezeigt wird. Dies wird meistens mit einer One-Hot-Kodierung bzw. einem One-Hot-Vektor gemacht. Jeder Labelart wird eine eigene Vektorkomponente zugewiesen. Falls das Label aktiv ist, ist die Komponent 1 ansonsten 0. Somit kann Hund als $\mathbf{l}_H = (1, 0)^T$ reprasentiert werden und Katze als $\mathbf{l}_K = (0, 1)^T$.

Abbildung 1: das beschriebene Machine Learning Modells

4.1.2 Modelle

Ein **Modell** ist eine mathematische Funktion $f\colon \mathbb{R}^n \to \mathbb{R}^m$, welche die Inputs auf die Outputs abbildet $\mathbf{y} = f(\mathbf{x})$. Im Zusammenhang mit Wertevorhersagen spricht man von einem Regressionsmodell. Das Verhalten dieses Modells wird bestimmt durch seine **Modellparameter** $\lambda_1, \lambda_2, \ldots$. Das Ziel ist es, die Modellparameter so einzustellen, dass die Vorhersagen $\mathbf{y}$ besser mit den Labels $\hat{\mathbf{y}}$ uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Parametern vorgenommen werden, bis sie das gewuenschte Resultat liefern.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang der Form $y = \lambda_1 x + \lambda_0$ zwischen den Features und den Labels besteht. Nun muessten nur noch λ_0 und λ_1 bestimmt werden.
Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen.

Fuer Machine Learning haben sich gewisse Modelle besonders guet etabliert, darunter waeren: Support Vector Machines, Evolutionäre Algorithmen, und Künstliche Neuronale Netze. Diese Arbeit wird sich vorwiegend mit Neuronalen Netzen auseinandersetzen.

BEISPIELMODELL GLEICHES WIE VORHIN

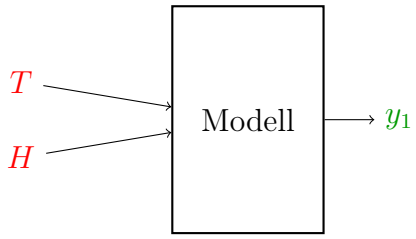

Abbildung 2: beschriebenes Beispielmodells

4.2 Training

4.2.1 Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Modells bewertet werden. Dafuer gibt es sogennante **Kosten-, Verlust- oder Fehlerfunktionen**. Sie sollen ein Mass fuer die Abweichung des Outputs $\mathbf{y}$ vom Label $\hat{\mathbf{y}}$ sein.

Die Fehler der einzelnen Outputs $c(y_i, \hat{y}_i)$ werden aufsummiert um den Fehler $C(\mathbf{y}_j, \hat{\mathbf{y}}_j)$ (wobei $\mathbf{y}_j$ die Vorhersagen sind und $\hat{\mathbf{y}}_j$ die Labels sind) einer gesamten Vorhersage zu berechnen. Siehe Gleichung (1).
Der Fehler $\bar{C}(\mathbb{T})$ eines ganzen Datensatzes $\mathbb{T}$, der Groesse p ergibt sich aus dem arithmetischen Mittel der einzelnen Vorhersagen-Fehler. Siehe Gleichung (2).

$$C(\mathbf{y}, \hat{\mathbf{y}}) = \sum_{i=1}^{m} c(y_i, \hat{y}_i) \qquad (1) \qquad\qquad \bar{C}(\mathbb{T}) = \frac{1}{p} \sum_{j=1}^{p} C(\mathbf{y}_j, \hat{\mathbf{y}}_j) \qquad (2)$$

Eine Kostenfunktion sollte folgende Eigenschaften aufweisen:

- c ist minimal, wenn $y = \hat{y}$
- c waechst mit $|\mathbf{y} - \hat{\mathbf{y}}|$
- C ist nach jedem y_n partiell differenzierbar (erklaert in Sektion (**??**))

Eine beliebte Kostenfunktion ist die "Mittlere quadratische Abweichung" (engl.: mean squared error).
$C_{MSE} = \frac{1}{2n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \frac{1}{2n} (\hat{\mathbf{y}} - \mathbf{y})^2$. Sie erfuellt alle anforderungen, denn: Sie ist 0 falls $y = \hat{y}$. Sie ist proportional zu $(\hat{y} - y)^2$ und ihre Ableitung nach y lautet: $C' = \frac{1}{n}(y - \hat{y})$

4.2.2 Gradientenverfahren

Um ein Modell nun zu trainieren, muss man verstehen, dass es sich um ein Optimierungsproblem handelt. Das Modell ist am besten, also macht die besten Vorraussagen, wenn die Funktionswerte der Fehlerfunktion am kleinsten sind. Deshalb muss diese Fehlerfunktion C minimiert werden. Hierbei muss die Funktion C nicht mehr in Abhaengigkeit der Inputs betrachtet werden, sondern in Abhaengigkeit von den Modellparametern $C(\lambda_1, \lambda_2, \ldots, \lambda_n)$, denn diese sollen angepasst werden, um das Modell zu verbessern. Fuer diese Optimiertung wird das sogennante **Gradientenverfahren** (engl.: Gradient descent) verwendet. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \ldots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ (wie z.B. die Fehlerfunktion) minimiert werden. Dies geschieht, indem ein Startpunkt (Ortsvektor) $\mathbf{p}_0$, dessen Komponenten den Input von f darstellt, gewaehlt wird. Nun werden iterativ neue Punkte $\mathbf{p_z}$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte, die den Funktionswert $f(\mathbf{p}_z)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genug nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\mathbf{b}_z$ bestimmt werden, welcher auf den Punkt $\mathbf{p}_z$ draufaddiert einen neuen Punkt $\mathbf{p}_{z+1}$ bildet, bei dem der Funktionswert $f(\mathbf{p}_{z+1})$ kleiner ist als der von $f(\mathbf{p}_z)$. Dies geschieht am effizientesten, wenn $\mathbf{b}_z$ in die Richtung der staerksten Funktionswertabnahme zeigt.
Hierzu braucht man den sogennanten **Gradient** ∇, wofür man wiederum partielle Ableitungen braucht.

Partielle Ableitungen

Partielle Ableitungen sind eine Erweiterung der "normalen" Ableitungen auf multidimensionale Funktionen $f(x_1, \cdots, x_n)$. Man leitet dabei nur nach einem Parameter x_i ab und betrachtet die restlichen Argumente als Konstanten. Es gelten die gleichen Ableitungsregeln wie bei der nicht-partiellen Ableitung. Die partielle Ableitung einer Funktion $f(x_1, \cdots, x_n)$ bezueglich der Variable x_i in einem Punkt $\mathbf{a} = (a_1, \cdots, a_n)^{\mathrm{T}}$ ist analog zur "normalen" Ableitung folgendermassen definiert:

$$\frac{\partial f}{\partial x_i}(\mathbf{a}) := \lim_{h \to 0} \frac{f(a_1, \cdots, a_i + h, \cdots, a_n) - f(a_1, \cdots, a_i, \cdots, a_n)}{h}$$

Geometisch ist dies die Steigung der Tangente im Punkt $\mathbf{a}$ in Richtung der Achse des Parametern x_i, nach dem man ableitet.

[1] Im Gymnasium wird beigebracht die lokalen Extrema (inkl. lokalen Minimas) zu bestimmen, indem die erste Ableitung f' gebildet wird und gleich null gesetzt wird. Dies ist hier nicht moeglich, da die Funktion $C'(\lambda_1, \lambda_2, \ldots, \lambda_n)$ deutlich zu komplex ist, um die Nullstellen zu bestimmen. Deshalb wird das Gradientenverfahren verwendet.

Gradient

Der Gradient ∇ ist ein Differentialoperator, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet.
Dabei fasst man alle partiellen Ableitungen einer Funktion f in einem Vektor zusammen.

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Geometisch ist der Gradient $\nabla f(\mathbf{p})$ einer Funktion f in einem Punkt $\mathbf{p}$ dann der Vektor, welcher in die Richtung des steilsten Anstiegs von f zeigt. Sein Betrag gibt die Staerke des Anstiegs an.

Da der Gradient in die Richtung des steilsten Anstiegs zeigt, sollte der Vektor $\mathbf{b}_z$ in die Richtung des negierten Gradient der Funktion f im Punkt $\mathbf{p}_z$ zeigen. Also kann jetzt das iterative Annahern an das lokalen Minimums so beschrieben werden (wobei η eine spaeter erklaerte Schrittgroesse darstellt):

$$\mathbf{p}_{z+1} = \mathbf{p}_z - \eta \cdot \nabla f(\mathbf{p}_z) \tag{3}$$

BEISPIEL FUER GRADIENTENVERFAHREN MIT ABBILDUNG

Abbildung 3: Visualisierung des Gradientenabstieg

Waehrend des Gradientenverfahren konvergiert der Punkt $\mathbf{p}_z$ zu einem belibigen *lokalen* Minimum, abhängig davon wie der Startpunkt $\mathbf{p}_0$ gewaehlt wurde. Da dieser meist zufaellig bestimmt wird, ist es eine Glückssache ein sehr niedriges Minimum zu finden.

Die sogennante **Lernrate** η aus Gleichung (3) ist ein Hyperparameter. Sie ist ein positiver Proportionalitatsfaktor, welcher die Schrittgroesse des Gradientenabstieg bestimmt. Sie muss je nach zu minimierender Funktion anderst gewaehlt werden. Dabei hilft nur ausporbieren. Falls η nicht gut gewahelt wurde, gibt es Probleme beim Training:

- Falls η zu klein ist, verlaueft das Trainings unnotig langsam und braucht sehr lange. Ausserdem kann es passieren, dass man bei einem hohen lokalen Minimum stecken bleibt.
- Falls η zu gross ist, passiert es, dass man ueber das lokale Minimum hinaus schiesst und nur darum herum springt.

4.2.3 Stochastisches Gradientenverfahren fuer Machine Learning

Wie vorhin erklaert, wird fuer das Trainieren eines Modells das Gradientenverfahren benutzt. Konkret wird die Kostenfunktion $C(\mathbf{y}, \hat{\mathbf{y}}|\lambda_0, \ldots, \lambda_n)$ (λ sind die Modellparameter, $\mathbf{y}$ die Vorhersagen und $\hat{\mathbf{y}}$ die Labels) minimieren, indem die Parameter angepasst werden. Dadurch macht das Modell immer bessere Vorhersagen. Fuer nur eine Iteration des Gradientenverfahren, muesste man den Gradienten fuer den *gesamten* Trainingsdatensatz berechnen. Dies waere zwar ein exakter Prozess, aber ein extrem langsamer zugleich. Bei grossen Datensaetzen wuerde es eine Ewigkeit dauern, bis das Modell nur annahernd guete Vorhersagen machen wuerde.

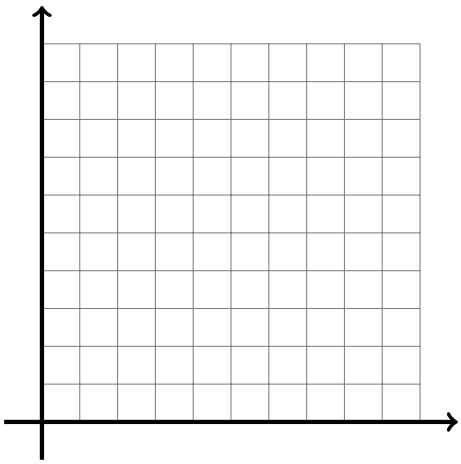

Abbildung 4: Auswirkungen verschiedener Lernraten

Aus diesem Grund verwendet man eine leicht abgeänderte Variante dieses Verfahren, naemlich das **Stochastische Gradientenverfahren** (engl.: Stochastic Gradient Descent). Hierfuer wird der "echte" Gradient des gesamten Datensatzes mit dem Gradienten einiger Trainingsbeispiel approximiert. Dazu wird der Trainingsdatensatz in sogennante **Mini-Batches** eingeteilt und der Gradient jeweils pro Mini-Batch berechnet. Als Konsequenz finden deutlich mehr Iterationen statt in *einer* Durchkaemmung der Trainingsdaten, welche man als **Epoche** bezeichnet. Oft wird mehrere Epochen lang trainiert. Der Gradient eines genug grossen Mini-Batches ist zwar nicht ganz exakt, aber approximiert den Gradieten des gesamten Datensatzen genuegend gut. Sowohl die Mini-Batch Groesse, wie auch die Anzahl Epochen sind weitere Hyperparameter.

Die partiellen Ableitungen der gesamten Trainingsdaten wird mit dem arithmetischen Mittel der partiellen Ableitungen eines Mini-Batches der Groesse q approximiert. Siehe Gleichung (4).

$$\frac{\partial \bar{C}}{\partial \lambda_k} \approx \frac{1}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_k} \tag{4}$$

Eine Iteration des Stochastischen Graidentenverfahren wird analog zu Gleichung (3) folgendermassen durchgefuehrt:

$$\lambda_{k,t+1} = \lambda_{k,t} - \frac{\eta}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_{k,t}} \tag{5}$$

4.2.4 Adam Optimizer

ZUSAMMENFASSUNG TRAININGSALGORITHMUS

4.3 Künstliche Neuronale Netze und Deep Learning

Das wohl beste Modell fuer die meisten Problemstellungen in Bereich Machinelles Lernen (Bilderkennung, Spracherkennung, etc.) ist das **Kuenstliche Neuronale Netz**, kurz KNN (eng.: Neural Network). Die Methoden welche benutzt werden um Neuronale Netze zu traniert fasst man unter dem Begriff **Deep Learning** zusammen.
Kuenstliche Neuronale Netze sind zum Teil biologisch inspiriert von Nervensystemen von Lebewesen. Sie sind aber lediglich eine Abstraktion der Informationverarbeitung und versuchen nicht eine moeglichst genaue biologische Abbildung darzustellen. Es gibt nicht nur eine Art von Neuralem Netz, sondern es exisitieren die verschiedensten Architekturen, welche je nach Problemstellung ausgewaehlt werden muessen. Diese Arbeit wird sich spaeter vorallem mit sogennanten Autoencodern auseinandersetzen.

4.3.1 Perzeptron

Um den Aufbau und die Funktion eines Kuenstlichen Neuronalen Netz besser zu verstehen, wird im folgenden ein Vorgaenger des KNN erklaert: das **Perzeptron**.

Das einlagige Perzeptron wurde erstmals 1958 von Frank Rosenblatt vorgestellt. Dieses besteht aus einem einzigen Kuenstlichen Neuron. Dieses Kuenstliche Neuron hat mehrere binaere Inputs und einen einzigen binaeren Output. Binaer bedeutet, dass der Wert nur entweder 0 (*aus*) oder 1 (*ein*) sein kann. Des weiteren besitzt es mehrere sogenannte **Gewichte** $w_1, \ldots, w_n \in \mathbb{R}$, fuer jeden Input ein Gewicht. Diese sind reelle Zahlen, welche das Verhalten des Perzeptron bestimmen. Die **gewichtete Summe**, also die Summe aller Produkte der Inputs mit ihrem Gewicht, wird mit $\tilde{z}$ bezeichnet. Sie ist das gleiche wie das Skalarprodukt des Gewichtevektor mit dem Inputvektor: $\tilde{z} = \sum_{i=1}^{n} w_i x_i = \mathbf{w} \cdot \mathbf{x}$
Zusaetzlich besitzt das Perzeptron einen **Schwellenwert** $\tilde{b}$. Zusammen mit den Gewichten, bildet er die Modellparameter. Das Perzeptron verhaelt sich so, dass falls die gewichtete Summe groesser als der Schwellwert ist, das Neuron feuert, d.h. der Output betraegt 1. Andernfalls ist er 0. Siehe erster Teil der Gleichung (6). Es ist gaengig die Ungleichung der Bedingung in die Nullstellenform zu bringen und $\tilde{b}$ durch die **Neigung** (engl.: Bias) $b = -\tilde{b}$ zu ersetzten. Somit lautet die Ungleichung: $\tilde{z} + b > 0$. Der Term $\tilde{z} + b$ wird mit z bezeichnet. Siehe Rest der Gleichung (6). Die Neigung gibt an wie stark das Neuron dazu neigt zu feuern. Ein hohes b laesst ein Neuron auch trotz einigen Nullen in den Inputs feuern, waehrend es fuer ein tiefes b nicht feuern lassen wuerde.

$$p(\mathbf{x}) = \begin{cases} 1 & \text{falls } \tilde{z} > \tilde{b} \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \tilde{z} + b > 0 \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{ansonsten} \end{cases} \tag{6}$$

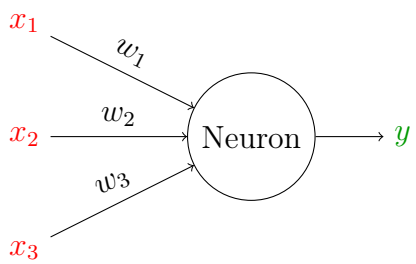

Abbildung 5: Perzeptron mit drei Inputs

Fuer das Trainieren des Perzeptron existieren spezielle Verfahren, welche hier aber nicht relevant sind. Das Gradientenverfahren kann naemlich nicht verwendet werden. Der Grund dafuer sollte spaeter in Sektion (4.3.3) einleuchtend werden.

Nun stellt sich die Frage, was ein Perzeptron erlernen kann und wofuer es genutzt werden kann. Das Perzeptron ist lediglich ein **Linearer Klassifikator** der Form $y = w_1x_1 + \cdots + w_nx_n$. Es kann die Features in zwei Klassen 0 oder 1 einordnen. Ueberschreitet y den Schwellenwert, werden die Features der Klasse 1 zugeordnet, sonst der Klasse 0. Jedoch muessen diese Klassen linear separierbar sein. Das bedeutet, dass die Inputvektoren $\mathbf{x}$ durch eine Hyperebene trennbar sein muessen. Bei zwei Features (zwei Inputs) handelt es sich bei der Hyperebene um eine Gerade welche die Vektoren auftrennt. Siehe Abbildung (6). Fuer drei Featuers ist es eine Ebene. ZU ABSTRAKT -> BEISPIEL
Eine Funktionen, welche nicht linear separierbar ist (z.B XOR-Verknuepfungen), kann von einem Perzeptron nicht erlernt werden.

Abbildung 6: erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts)

(**wiki:perzeptron**)

4.3.2 Biologische Analogie

SCHREIBEN

4.3.3 Erweiterung der Kuenstlichen Neuronen

Ein Perzeptron ist, wie vorhin erklaert, nur in der Lage, lineare Klassifikationen durchzufuehren. Um nun auch kompliziertere Probleme zu loesen, muss das Prinzip ausgebaut werden. Ausserdem brauchen wir ein Kuenstliches Neuron, welches sich besonders gut als Baustein fuer KNNs eignet.

4.3.3.1 Kuenstliche Neuronen im Allgemeinen

Kuenstliche Neuronen sind immer so aufgebaut, dass sie einen oder mehrere Inputs haben und einen einzigen Output. Zu jedem Input x_i ist ein Gewicht w_{ji} assoziert. Zuerst wird die gewichtete Summe der Inputs $\tilde{z}$ gebildet. Die Neigung b wird ebenfalls draufaddiert, um z zu erhalten. Nun muss die sogennante Aktivierung a gebildet werden. Sie entspricht dem Output des Neurons. Die Aktivierung a ist das Resultat der Aktivierungsfunktion φ angewendet auf z. $a = \varphi(z)$. Die verschiedenen Kuenstlichen Neuronen unterscheiden sich fast nur in ihrer Aktivierungsfunktion.

4.3.3.2 Perzeptronen als Kuenstliche Neuronen

Zuerst noch einmal ein Blick auf das Perzeptron in Betracht der Aktivierungsfunktion. Ein wesentlicher Unterschied des Perzeptron gegenueber sonstigen Kuenstlichen Neuronen, besteht darin, dass seine Inputs und Outputs nur binaere Werte annehmen koennen. Um dieses Verhalten des Perzeptron zu erhalten, muss eine Stufenfunktion als Aktivierungfunktion verwendet werden: die Heaviside-Funktion Θ. Sie hat einen einzigen Stufensprung bei $x = 0$ vom Wert 0 auf 1. Siehe Abb. (7).

$$\varphi^{\text{hlim}}(z) = \Theta(z) = \begin{cases} 1 & \text{falls } z \geq 0 \\ 0 & \text{falls } z < 0 \end{cases}$$

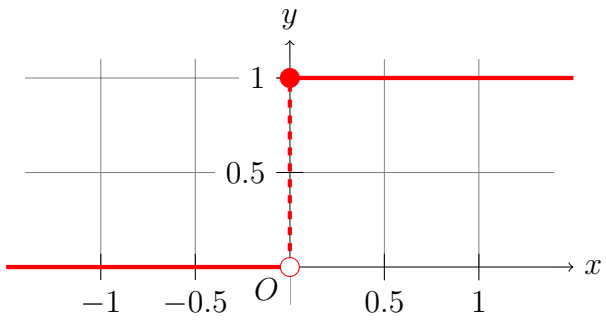

Abbildung 7: Definition und Graph der Heaviside-Funktion Θ

4.3.3.3 Stueckweise lineare Neuronen

Der naechste Schritt nach dem binaeren Perzeptron sind sogenannte stueckweise lineare Neuronen. Sie verwenden eine stueckweise lineare Aktivierungsfunktion. Diese bildet ein beschraenktes Intervall linear ab. Die Werte ausserhalb werden auf die konstanten Werte 0 oder 1 abbgebildet. Siehe Abb. (8).

Die Inputs koennen jetzt beliebige reelle Zahlen sein (vorzugsweise in der Naehe von 0 und 1). Ein KNN aus ausschliesslich linearen Neuronen ist ziemlich nutzlos, da jedliche Verkettung von linearen Funktion auch durch eine einzige lineare Funktion dargestellt werden koennte. Somit hat ein KNN gegenueber einem einzelnen Neuron keinen Mehrwert. Ausserdem koennen mit diesen Neuron auch nur lineare Probleme geloest werden.

$$\varphi^{\text{pwl}}(z) = \begin{cases} 1 & \text{falls } z > \frac{1}{2} \\ z + \frac{1}{2} & \text{falls } -\frac{1}{2} < z < \frac{1}{2} \\ 0 & \text{falls } z < \frac{1}{2} \end{cases}$$

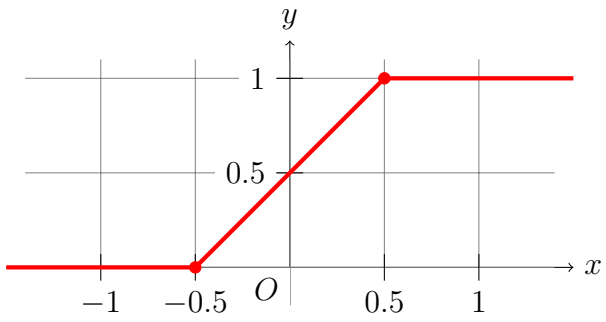

Abbildung 8: Formel und Graph einer stueckweisen linearen Funktion

4.3.3.4 Sigmoide Neuronen

Die logische naechste Erweiterung sind nicht-lineare Neuronen. Ein sehr beliebter Kandidat dafuer sind Sigmoide Neuronen. Den Namen haben sie von ihrer Aktivierungsfunktion: der Sigmoidfunktion σ. Eine sehr wichtige Eigenschaft von ihr ist, dass sie - im Gegensatz zu den vorhin genannten Aktivierungsfunktion - ueberall differenzierbar und strikt monoton steigend ist. Erst fuer diese Aktivierungsfunktion, kann das Gradientenverfahren angewendet werden und somit das KNN trainiert werden. Die anderen Funktionen haben Stellen bei welchen die Ableitungsfunktion null betraegt, somit kann das lokale Minimum nicht gefunden werden. Nicht nur die Sigmoidfunktion erfuellt diese Bedingungen, sondern auch andere. Die Sigmoidfunktion wird bevorzugt, da ihre Ableitung sehr simpel ist, siehe Abb. (9). Die Nicht-linearitaet dieser Neuronen ermoeglicht ein Erlernen von deutlich komplexeren Sachverhalten. Und wie spaeter in Sektion (4.3.8) erlautert, ist es moeglich mit der Kompostion von nicht-linearen Funktionen jede belibige Funktion zu approximieren!

Die Simgmoid-Funktion besitzt eine einzige Wendestelle bei $x = 0$ und hat zwei Asymptoten, eine fuer $x \to -\infty$ mit $y = 0$ und eine zweite fuer $x \to \infty$ mit $y = 1$. Siehe Abb. (9).

$$\varphi^{\text{sig}}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

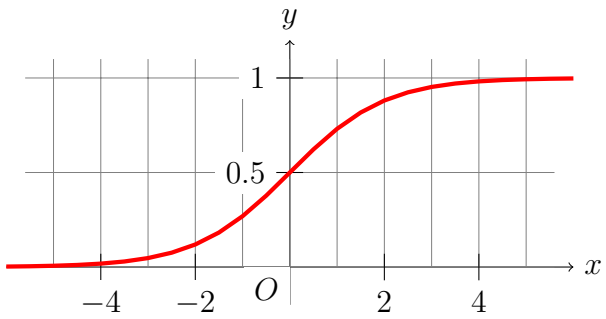

Abbildung 9: Definition, Ableitung und Graph der Sigmoid-Funktion σ

4.3.4 Topologie der Kuenstlichen Neuronalen Netzen

Nun sollten diese Sigmoiden-Neuronen als Bausteine verwendet werden, um ein Kuenstliches Neuronales Netz zu bilden. Dazu werden sie miteinander verbundet und bilden so ein Netz, aehnlich wie ein Nervensystem. Diese Neuronen sind in verschieden Schichten (engl.: Layers) arangiert. Die erste ist die **Inputschicht**. Sie behinhalten die Inputneuronen. Dies sind eigentlich keine richtigen Neuronen sondern eher Platzhalter fuer ihren jeweiligen Inputwert x_i. Als letztes kommt die **Outputschicht** mit den Outputneuronen, welche jeweils einen Outputwert y_i besitzen. Dazwischen liegen die **Zwischenschichten** (engl.: Hiddenlayer). Von ihnen kann es beliebig viele geben und in ihnen beliebig viele Neuronen. Der Aufbau eines KNN bezeichnet man als **Topologie** des Netzes. Die Topologie umfasst viele Hyperparameter. Darunter sind zum Beispiel die Anzahl Zwischenschichten, wie auch die Anzahl Neuronen pro Schicht.

Jedes Neuron aus einer Schicht ist mit jedem Neuron aus der naechsten Schicht ueber Verbindungen gekoppelt. Alle Verbindungen besitzten ein Gewicht analog zu den Inputs des Perzeptron. Die Aktivierung, also der Output, eines Neurons wandert entlang den jeweiligen Verbindung zu allen Neuronen der naechsten Schicht und dient als deren Input. Die soeben beschriebe Art von KNN nennt man **Feedforward-Netz**, da alle Werte ausschliesslich nach vorne propagiert werden.
In Abbildung (4.3.5) ist ein Beispiel eines Neuronalen Netzes abgebildet. In diesem Fall besitz es sowohl 4 Inputs, wie auch 4 Outputs. Es hat ausserdem 3 Zwischenschichten. Die zwei aueserren Zwischenschichten haben 3 Neuronen und die dazwischen hat sogar 4 Neuronen.
ABBILDUNG MIT BESCHRIFTUNG DER GEWICHTE

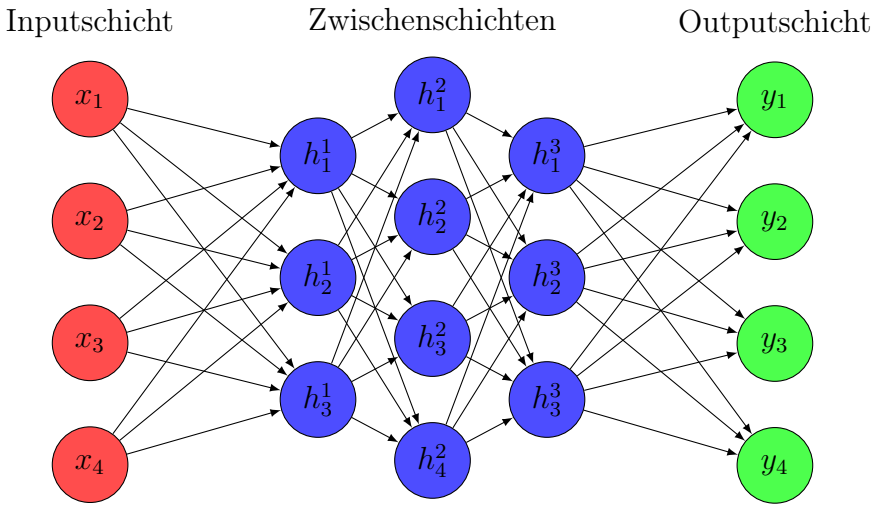

Abbildung 10: Schichten eines KNNs

4.3.5 Vorwaertspropagierung

Jetzt, da der Aufbau eines KNNs erklaert wurde, sollte nun die mathematische Funktionsweise des Modells erklaert werden. Hierfuer muessen einige Konventionen zur Bezeichnung der Teile eines KNNs getroffen werden:

- n_j^l ist das j-te Neuron in der l-ten Schicht.
- z_j^l ist die gewichtete Summe der Inputs des j-ten Neuron in der l-ten Schicht.
- a_j^l ist die Aktivierung des j-ten Neurons in der l-ten Schicht.
- b_j^l ist die Neigung des j-ten Neuron in der l-ten Schicht.
- $w_{j,k}^l$ ist das Gewicht der Verbindung vom k-ten Neuron in der $(l-1)$-ten Schicht zum j-ten Neuron in der l-ten Schicht (man beachte die Reihenfolge) [2].
- L soll die gesamte Anzahl der Schichten sein.
- N_l ist die Anzahl Neuronen in der l-ten Schicht.
- φ ist die gewaehlte Aktivierungsfunktion (grundsaetzlich ist diese immer die Sigmoidfunktion σ).

ABBILDUNG MIT GEWICHTEBESCHRIFTUNGEN (KORREKT!)

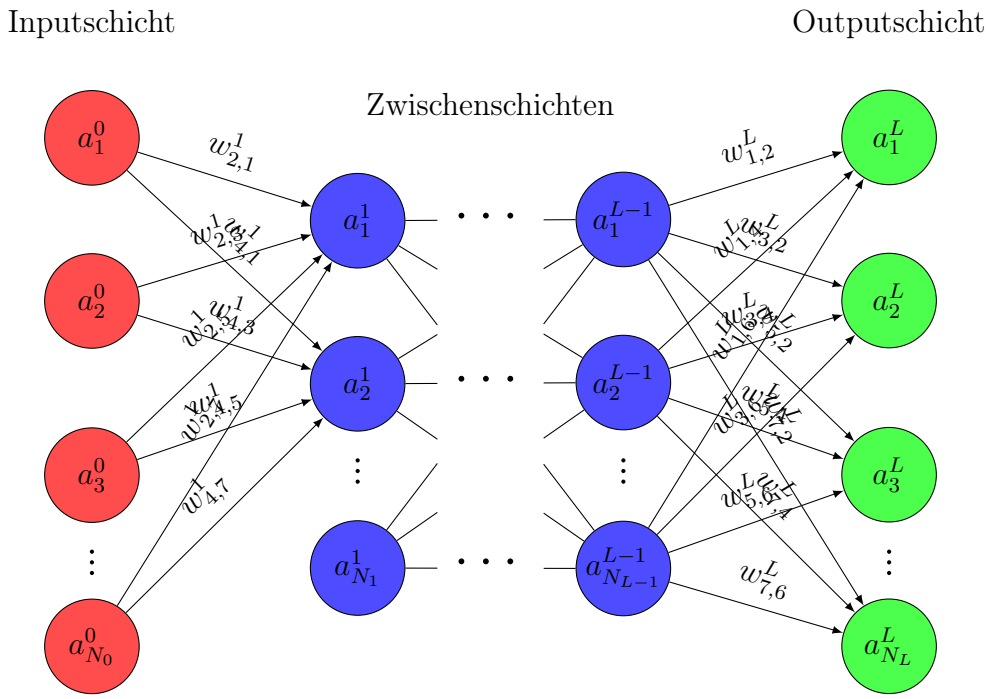

Abbildung 11: zum Verstaentniss der Nomenklatur

Die Vorwaertspropagierung beginnt bei den Inputneuronen, welche jeweils einen Inputwert in sich tragen. Diese werden, um fuer eine kohaerente Nomenklatur zu sorgen, gleich wie die Aktivierungen der anderen Neuronen mit a_j^0 bezeichnet, wobei j der Index des Neurons ist.

[2] Diese Konvention scheint auf den ersten Blick unintuitiv, macht jedoch Sinn fuer die spaeteren mathematischen Operationen mit Matrizen in Sektion (4.3.7).

Nun muessen die restlichen Aktivierungen der Neuronen berechnet werden. Dies geschieht rekursiv, anhand der Aktivierungen der voherigen Schicht. Und zwar folgendermassen (ersichtlich in Gleichung (7)):
Zuerst laueft eine Summe ueber alle Neuronen n_k^{l-1} der voherigen Schicht $(l-1)$. Dabei wird die gewichtete Summe der Aktivierungen a_k^{l-1} mit den assozierten Gewichten $w_{j,k}^l$ gebildet. Hierbei ist das Gewicht jenes, welches das k-te Neuron der $(l-1)$-ten Schicht mit dem j-ten Neuron der l-ten Schicht verbindet. Zusaetzlich gehoert zu der gewichteten Summe auch die jeweilige Neigung b_j^l, welche dazuaddiert wird. Diese gewichtete Summe wird mit z_j^l bezeichnet.

$$z_j^l = \sum_{k=1}^{N_l} w_{j,k}^l a_k^{l-1} + b_j^l \tag{7}$$

Auf diese Summe wird dann die Aktivierungsfunktion φ angewandt. Das ist dann die Aktivierung a_j^l des Neurons.

$$a_j^l = \varphi\left(\sum_{k=1}^{N_l} w_{j,k}^l a_k^{l-1} + b_j^l\right) = \varphi\left(z_j^l\right) \tag{8}$$

Da KNNs weit verbreitet sind in der Bilderkennung und Spracherkennung, ist es nicht unueblich, dass diese sehr viele Neuronen und Verbindungen (ueber 100'000) besitzen. Um hierbei nicht den Ueberblick zu verlieren und um nicht in den Indizes zu ertrinken, greift man auf **Lineare Algebra** zurueck. Man verwendet Matrizen und Vektoren um die vielen Variabeln zusammen zu fassen. Ausserdem besteht ein weiterer Vorteil darin, dass Computer mithilfe von Vektor- und Matrixoperationen die Berechnungen parallisieren koennen und in kuerzerer Zeit und mit weniger Ressourcen viele Brechnungen gleichzeitig ausfuehren koennen. Dies beschleunigt das Training der Modelle um ein Vielfaches. Dies wird spaeter in Sektion (**??**) noch weiter thematisiert.

Die Inputs $\mathbf{x}$, Outputs $\mathbf{y}$ und Labels $\hat{\mathbf{y}}$ haben wir schon von Anfang an als Vektoren geschrieben. Nun sollen noch die Parameter und die restlichen Komponenten eines KNNs als Vektoren und Matrizen geschrieben werden. Sowohl alle gewichteten Summen z_j^l, wie auch alle Aktivierungen a_j^l einer Schicht l, werden in Vektoren $\mathbf{z}^l$ und $\mathbf{a}^l$ zusammengefasst. Alle Neigung b_j^l einer Schicht l bildet ebenfalls eines Vektor $\mathbf{b}^l$. Zu guter letzt, wird noch eine **Gewichtsmatrix** $\mathbf{W}^l$ fuer jede Schicht l definiert. Die Eintraege dieser Matrix sind einfach die Gewichte welche *zu* den Neuronen der l-ten Schicht verlaufen. Das heisst der Eintrag in der j-ten Zeile und in der k-ten Spalte ist $w_{j,k}^l$ (Siehe Gl. (9)).

$$\mathbf{W}^l = \begin{pmatrix} w_{1,1}^l & w_{1,2}^l & \cdots & w_{1,n}^l \\ w_{2,1}^l & w_{2,2}^l & \cdots & w_{2,n}^l \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1}^l & w_{m,2}^l & \cdots & w_{m,n}^l \end{pmatrix} \tag{9}$$

Mit diesen Definitionen kann nun Gleichung (8) in Matrixform geschrieben werden. Hierfuer muss noch ein weiteres mathematisches Konzept eingefuehrt werden: die Vektorisierung einer Funktion.

Vektorisierung einer Funktion

Die Vektoriserung einer Funktion f, geschrieben als $f[\mathbf{v}]$ ist eine neue Funktion, welche als Rueckgabewert einen Vektor hat, auf dessen Komponenten jeweils immer die Funktion f angewendet wird.

$$f[\mathbf{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

Nun kann die Matrixschreibweise eingefuehrt werden (siehe Gleichung (11)).
Zuerst wird eine Matrixmultiplikation der Gewichtsmatrix $\mathbf{W}^l$ mit dem Aktivierungsvektor $\mathbf{a}^{l-1}$ der vorherigen Schicht durchgefuehrt. Darauf wird der Neigunsvektor addiert. Und zu guter Letzt wird die vektorisierte Aktivierungsfunktion φ angewandt.

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l \tag{10}$$

$$\mathbf{a}^l = \varphi\left[\mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l\right] = \varphi\left[\mathbf{z}^l\right] \tag{11}$$

4.3.6 Gewichte und Neigungen initalisieren*

4.3.7 Rueckwaertspropagierung

Die wahre Herausforderung besteht beim Gradientenverfahren darin, die partiellen Ableitungen der Modellparameter, also die Komponenten des Gradienten, zu bestimmen. Anderst gesamt muessen alle $\frac{\partial C}{\partial w^l_{j,k}}$ wie auch alle $\frac{\partial C}{\partial b^l_k}$ berechnet werden. Das Verfahren zum bestimmen dieser Ausdruecke ist so spezfisch und aufwendig, dass das Gradientenverfahren fuer KNNs einen eigen Namen hat: die **Rueckwaertspropagierung** (engl.: Backpropagation).
Das Ziel dieses Algorithmus ist es die beiden Partiellen Ableitungen $\frac{\partial C}{\partial w^l_{j,k}}$ und $\frac{\partial C}{\partial b^l_j}$ zu bestimmen. Hierfuer ist es sinnvoll eine Zwischenwert, welchen wir Fehler δ^l_j nennen.

Hadamard-Produkt

Das Hadamard-Produkt ist ein spezielles Produkt zweier gleichgrossen Matrizen. Die resultierende Matrix ergibt sich aus der elementweisen Multiplikation der Ausgangsmatrizen.

$$\mathbf{A} \odot \mathbf{B} = \begin{pmatrix} a_{1,1}b_{1,1} & \cdots & a_{1,n}b_{1,n} \\ \vdots & \ddots & \vdots \\ a_{m,1}b_{m,1} & \cdots & a_{m,n}b_{m,n} \end{pmatrix} \in \mathbb{R}^{m \times n} \qquad \mathbf{v} \odot \mathbf{w} = \begin{pmatrix} v_1 w_1 \\ \vdots \\ v_n w_n \end{pmatrix}$$

$$\delta^l_j := \frac{\partial C}{\partial z^l_j} \tag{12}$$

Viel Kettenregel.

$$\frac{\partial C}{\partial w_{j,k}} = \frac{\partial C}{\partial a_k} \frac{\partial a_k}{\partial z_k} \frac{\partial z_k}{\partial w_{j,k}} \tag{13}$$

Die vier magischen Gleichungen:
Der Fehler in der Outputschicht

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L) \tag{BP1}$$

Matrixform

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \tag{BP1a}$$

Das tiefgestellte $_a$ des Nablas, gibt an, dass es sich um den Gradienten mit den partiellen Ableitungen bezueglich den Aktivierungen a handelt.
Fehler anhand des Fehlers der naechsten Schicht

$$\delta^l = ((w^{l+1})^{\mathrm{T}} \delta^{l+1}) \odot \sigma'(z^l) \tag{BP2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

$$\frac{\partial C}{\partial w_{j,k}^l} = a_k^{l-1} \delta_j^l \tag{BP4}$$

4.3.8 Universal Approximation Theorem

Neural Network kann alles erlernen

4.4 Convolutional Neural Networks

(**deeplearning.ai:cnn**) Viele Anwendungen von Machine Learning sind verbunden mit Bild- oder Audioverarbeitung, wie z.B Bildklassifizierung, Gesichterkennung oder Spracherkennung. Vorallem aber fuer hochaufloesende Bilder sind die KNNs, die wir soeben kennengelernt haben, nicht geeignet. Sie sind zum Teil gar nicht in der Lage eine Korrelation zwischen den Inputs und Outputs zu erlernen. Um diesen Umstand zu erklaeren, wird nun ein kleines Beispielmodell erlauetert:

Es soll ein KNN designed werden, welche einer Photographie klassifiezieren soll, ob ein Hund darauf sichtbar ist oder nicht. Wir nehmen fuer dieses Beispiel bereits ein relativ niedrig aufgeloestes Bild mit 256×256 Pixel, dies entspricht weniger als 0.07 Megapixel (ein iPhone XS hat eine Kamera mit 12 Megapixel). Um Farben darzustellen besitzt jeder Pixel drei Komponten R, G und B. Somit hat dieses Bild insgesamt $256 \times 256 \times 3 = 196'608$ Komponenten. Jede Komponente ist ein Feature fuer dieses KNN. Somit bestuende die erste Schicht des Netzwerkes aus fast $200'000$ Neuronen. Um diese Schicht nun mit seiner Nachbarsschicht, welche gleiche Dimensionen besitzt, zu verbinden, braucht es $196'608 \times 196'608 = 38'654'705'664$ Verbindungen und damit gleich so viele Gewichte! Fuer ein Netwerke ohne eine einzige Zwischenschichten gaebe es also ueber 38 Milliarden Modellparameter zu erlernen! Das dies nicht realistisch ist, sollte auf der Hand liegen.

Nun sollte klar sein, dass eine andere Modellarchitektur noetig ist, um Machine Learning auf Bilder anzuwenden. Fuer genau solche Anwendungen wurde eine modifizierte Version eines KNNs entwickelt: das **Convolutional Neural Network** (CNN). Diese Art von Netzwerk, macht Gebrauch von Konzepten aus der klassischen Bildverarbeitung, wie sie mit beispielsweise Photoshop gemacht werden kann. Wie beim Perzeptron und bei den klassischen KNNs auch, ist die Architektur biologisch inspiriert. Der folgende Abschnitt wird die Funktionsweise eines solchen CNNs erklaeren.

4.4.1 Bilder als 3D-Matrizen

Bei der Betrachtung von Bildern fuer Machine Learning, ist es sinnvoll, diese nicht als eine Anordnung von verschieden farbigen Pixeln zu interpretieren, sondern sie als **3D-Matrizen** zu untersuchen.

3D-Matrix

Eine 3D-Matrix ist ein mathematischen Objekt und eine Erweiterung von Matrizen fuer 3-dimensionale Zahlenanordnung. Vorstellen kann man sich eine 3D-Matrix als ein Volumen dass Zahlen in einem Raster angeordnet hat. Die Zahlen sind dabei die Elemente der Matrix. Analog zum Volumen, bezeichnet man die Form der Matrix mit Hoehe, Breite und Tiefe. Eine Matrix von der Form $3 \times 3 \times 3$ koennte folgendermassen aussehen:
TIKZPICTURE EINER 3D-Matrix

Ein Bild kann somit einfach als 3D-Matrix, bezeichnet mit $\mathbf{B}_{h \times w \times c}$ der Form (Bildhoehe $\times$ Bildbreite $\times$ Anzahl Farbkomponenten) betrachtet werden. Die Elemente der Matrix nehmen dann einfach die Werte der Pixelkomponenten an. Ein schwarzweiss Bild hat nur eine Komponente, welche die Helligkeit angibt. Siehe Abb. (12).

Eine solche **Bildmatrix** stellt dann den Input eines CNNs dar und nicht mehr wie voher ein Vektor.

Abbildung 12: Beispiel einer Bildmatrix

Abbildung 13: 2-dimensionalen Schicht von Neuronen

4.4.2 Filter in der Bildverarbeitung

Filter sind in der Bildverarbeitung sehr verbreitet. Jeder kennt sie entweder von Photoshop, von Instagram oder sonstigen Bildbearbeitungsprogrammen. CNNs machen Gebrauch von solchen Filtern um die Features eines Bildes zu erlernen. Zuerst werden wir 2D-Filter betrachten, welche sich fuer graustufige Bilder $\mathbf{B}_{h\times w}$ eignen.

Filter sind einfach Regionen, deutlich kleiner als das verarbeitete Bild, welche ueber alle Pixel wandert und diese manipuliert und so wieder ein neues Bild $\mathbf{B}^*$ liefert. Mathematisch gesehen handelt es sich bei einem Filter einfach um eine Matrix, die sogennante **Filtermatrix** oder Faltungsmatrix (siehe Abb. (14)), bezeichnet mit $\mathbf{F}_{f\times f}$. Solche Filter sind immer quadratisch, wobei ihre Zeilen- bzw. Spaltenlaenge mit f bezeichnet wird. Dabei ist f immer eine ungerade Zahl, damit ein Element immer im Zenturm des Filters liegt.

Abbildung 14: eine Filtermatrix

Das Verhalten des Filters wird durch seine Matrixeintraege bestimmt. Die Elemente koennen also so gewaehlt werden, dass wenn der Filter auf ein Bild angewandt wird, er bestimmte Features aus dem Bild heraushebt. Solche Features koennten zum Beispiel Raender oder Kanten sein, welche hervorgehoben werden.
-> Pixel sind wichtig im Kontext mit ihren Nachbarn (lokalisierte Features!) -> nicht jeder Pixel wichtig Position Rotation und Format koennen sich aendern -> Invarianz gewuenscht

Der Sobel-Filter ist ein typischer Kantendetektions-Filter mit einer Groesse von $f = 3$.

Abbildung 15: Sobel-Filter angewandt auf Beispielsbild

4.4.3 Filter als Faltungen (Convolutions)

ERWAEHNUNG VON CONVOLUTIONS
[3]

Als Beispiel nehmen wir eine Filtermatrix $\mathbf{F}_{3\times 3}$, bei welcher $f = 3$ gewaehlt wurde. Meistens beginnt der Filter oben links ueber das Bild $\mathbf{B}$ zu wandern. Der Filter wird so plaziert, dass er ueber einer (3×3)-Region des Bildes liegt. Diese Region nennt man das Rezeptive Feld des Filters. Es ist eine Untermatrix $\tilde{\mathbf{B}}_{3\times 3}$ der Bildmatrix $\mathbf{B}$, welche gleich gross wie der Filter ist. Diese Untermatrix wird mit $\tilde{\mathbf{B}}^{(2,2)}$ bezeichnet, wobei die hochgestellten Indizes $(2, 2)$ die Position des Elements im Zentrum der Untermatrix innerhalb der Obermatrix $\mathbf{B}$ darstellt.

Da nun jedem Element des Filters ein Element der Bildmatrixregion zugeordnet ist, koennen diese jetzt verrechnet werden. Es wird das Hadamard-Produkt (das elementweise Produkt) des Filters $\mathbf{F}$ mit der Bildmatrixregion $\tilde{\mathbf{B}}^{(2,2)}$ gebildet, welche eine neue Matrix $\mathbf{P}_{3\times 3} = \mathbf{F} \odot \tilde{\mathbf{B}}$

[3] Vor Convolution Operation: Neure Matrix bilden aus Filter, indem Filtermatrix vertikal und horizontal gespiegelt wird. Und erst mit dieser neuen Matrix wird dann das Hadamard-Produkt usw. gebildet
Das wird im Machine learning nicht gemacht. Deshalb ist es mathematisch gesehen keine richtige Convolution sondern ehere eine cross-correlatoin.

zurueckgibt. Nun werden alle Elemente der neuen Matrix $\mathbf{P}$ aufaddiert. Diese Zahl ist dann der Grauwert des ersten Pixels des bearbeiteten Bildes $\mathbf{B}^*$.

$$(\mathbf{B}^*)_{1,1} = \sum_{y=1}^{3}\sum_{x=1}^{3}(\mathbf{P})_{y,x}$$

Nun wird der Filter um ein Element nach rechts verschoben und die gleiche Prozedur angewendet, um den zweiten Pixel des neuen Bildes zu berechnen. Dies wird durchgezogen, bis die ganze erste Zeile der Bildmatrix durchstreift wurde. Danach wird der Filter wieder nach ganz rechts verschoben und er bewegt sich ein Element nach unten. Das geschieht, bis das ganze Bild verrechnet wurde. Dabei werden die Elemente des urspruenglichen Bildes durchaus mehrmals verrechnet, da es zu einer Ueberlappung des vorherigen Filter und des verschobenen Filters kommt.

Abbildung 16: Schema, wie ein Filter ueber ein Bild laueft

Es ist zu bemerken, dass das neue Bild $\mathbf{B}^*_{(h-2)\times(w-2)}$ nicht mehr die gleichen Masse hat, wie das Ursprungsbild $\mathbf{B}_{h\times w}$. BEGRUENDUNG

Hier nochmals alle Schritte fuer allgemeine Masse zusammengefasst:
NOCH ALLGEMEINERE GLEICHUNG -> BESCHRIFTUNG (REGION ANGEBEN)

4.4.4 Filter fuer farbige Bilder

$$\mathbf{P}^{(y,x)}_{f\times f} = \mathbf{F}_{f\times f} \odot \tilde{\mathbf{B}}^{(y,x)}_{f\times f} \quad \forall y \in \{1,\dots,h\}, \forall y \in \{1,\dots,w\} \tag{14}$$

$$(\mathbf{B}^*_{(h-1)\times(w-1)})_{y,x} = \sum_{i=1}^{f}\sum_{j=1}^{f}(\mathbf{P}^{(y,x)})_{i,j} \tag{15}$$

4.4.5 Filter in CNNs

Tiefe des Filters muss gleich sein wie Tiefe des Bildes (Anzahl Channels)
Input bereich eines Neurons wird als Receiptive FIeld bezeichnet. Wahrnehmungsfeld. Nicht wie bei fully-connected Layer. Receiptive FIeld ist Hyperparameter
Filter wird gelernt F = (w11, w12, w13, ...) Matrix. Die Elemente der Filtermatrix sind die Modellparameter.
Idee der Convolution anhand von Randerkennung (Edge Detection).
Detecting vertical edges. and horizontal edges

4.4.5.1 Padding

Durch Convolution schrumpft Bild (am Rand). M(n x n) conv M(f x f) = M(n-f+1 x n-f+1). Problem nach ein paar Convolution ist Bild extrem klein -> M1x1 2. Problem: Viel Overlapping in der Mitte (Bildmatrix Elemente mehrmals genutzt) an Raender jedoch nicht! -> Information verloren von den Raendern des Bildes Darum Padding (vor der Convolution)! Zusaetzlicher Rand hinzugefuegt zu Bild vor der Konvolution. Zusatz Elemente haben Wert 0. Padding = 1 bedeutet an allen Raendern wird eine Reihe hinzugefuegt. Ziel ist es Dimensionen beizubehalten Begrifflichkeit: 'valid'-convolution: kein padding 'same'-convolution: mit padding, so dass Groesse beibehalten wird Rechnung fuer outputdimension bei same: m = n + 2p - f + 1 => p = (f-1)/2
f ist normalerweise eine ungerade Zahl, ansonsten waere das Padding unsymmetrisch.

4.4.5.2 Stride

Stride ist Schrittgroesse der Filterbewegung -> Stride = 2 bedeutet, dass ein Schritt uebersprungen wird. Outputmatrix wird kleiner
Rechnung fuer Outputdimension: m = (n + 2p - f)/s + 1; Abgerundet falls keine ganze Zahl

4.4.5.3 Mehrere Filter

Beispiel mit 2 Filter. Filter1 wird angewandt auf Bild -> M1 Filter2 wird angewandt auf Bild -> M2 Beide Matrizen M1 und M2 werden hintereinander gestellt und formen einen 3D-Tensor mit Dimension (fxfx2). Dieses Volumen kann dann erneut Convoluted werden.
Hierfuer muessen alle Filter gleich gross sein.
Zusammenfassung Rechnung:
M(n x n x c) conv M(f x f x c) = M(n-f+1 x n-f+1 x anzahlfilter)

4.4.6 Convolutional Layer

Bild ist Matrix (h x w x c) Bild wird du 3D-Tensor mit Tiefe 3 fuer R,G und B 3D Filter noetig (f x f x c). Tiefe des Filters und des Bildes muss gleich sein. Output ist dann 2D-Matrix, da Filter nur entland der Hoehe und Breite des Bildes bewegt wird und nicht der Tiefe entlang, da die Tiefe des Filters gleich der Tiefe des Bildes ist.
Schritte: Convolution wird angewandt. Auf Outputmatrix wird Bias (Skalar) draufaddiert (auf jedes Element der gleiche Bias) Jeder Filter hat seinen eigenen Bias Nicht-lineare Aktivierungsfunktion wird auf Matrix angewandt Nun werden die verschieden Outputmatrizen der Filter ausgestackt zu einem 3D-Tensor fertig!

4.4.7 Pooling Layer

Pooling reduziert die Dimensionalitaet der Daten, indem eine Gruppe von Neuronen der vorherigen Schicht zu einem einzigen Neuron der naechsten Schicht zusammengefasst wird.
Local Pooling 2x2 Feld wird zu einem Neuron zusammengefasst. MaxPooling -> behaelt einfach maximalen Wert des Feld bei und verwirft den Rest. Average Pooling berechnet das arithmethische Mittel des Feldes und verwednet dieses. MaxPooling ist am effektivsten.
Pooling: Komprimierung des Bildes. Durchschnitt von 2x2 Pixeln berchnen -> ein Pixel des gepoolten Bild. -> 75% Informationsreduktion .
Pooling als Filter betrachtet: MaxPooling mit 2x2 Feld. Filtergroesse $f = 2$ und stride $s = 2$. Dies sind die Hyperparamter des MaxPooling. Hat also zwar Hyperparameter aber keinerlei Modellparameter
MaxPooling koennte auch andere Filtergroesse oder Stride haben. Formeln fuer Outputgroesse gelten auch hier.
Bei 3D-Tensoren aendert sich die Depth nicht, da das Pooling auf jeden Channel einzeln angewandt wird.
Es gibt zwei Konventionen: Entweder bezeichnet man die Einheit von einem ConvLayer und einem PoolLayer zu einer Schicht zusammengefasst werden oder sie werden als seperate Schichten betrachtet...

4.4.8 Fully-connected Layer

4.4.9 Das Netzwerk und sein Aufbau

Ein klassisches CNN besteht aus folgenden drei Typen von Schichten: Convolutional Schichten, Pooling Schichten und den bereits bekannten fully-connected Schichten. Zuerst kommt immer

Abbildung 17: Schema eines CNNs

eine oder mehrere Convolutional Schichten, direkt darauf folgt dann eine Pooling Schicht. Dieses Muster kann dann beliebig oft wiederholt werden, bis die gewuenschte Komplexitaet erreicht wurde. Bei Bedarf koennen vor oder hinter einem solchen Muster eine fully-connected Schicht eingefuegt werden.
Beschriftung: Inputbilder werden wie Aktivierungen mit a^l bezeichnet. Die Filter sind aehnlich zu den Gewichten und werden mit W^l bezeichnet. Und der Bias bleibt aehnlich zum alten Bias b.
Konventionen zur Beschriftung: $f^{[l]}$ ist Filter Groesse
$p^{[l]}$ ist das padding
s^l ist stride
Input-Dimension ist $n_H^{[l-1]} \times n_W^{[l-1]} \times n_C^{[l-1]}$
Output-Dimension ist $n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$

$n[l] = \frac{n^{[l-1]}+2p^{[l]}-f^{[l]}}{s^{[l]}} + 1$ jeweils fuer Hoehe und Breite

n_C^l ist Anzahl Filter

Filter-Dimension ist: $f^{[l]} \times f^{[l]} \times n_C^{[l-1]}$
Aktivierungen $a^{[l]}$ -> $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
Gewichte $w^{[l]}$ -> $f^{[l]} \times f^{[l]} \times n_C^{[l-1]} \times n_c^{[l]}$ letzter Term ist ja gleich der Anzahl Filter
Vektor Bias: $n_c^{[l]}$
SchichtIndizes der Filter,Padding und stride sind gleich den Indizes des Output(aktivierungen)
Flattening ist, wenn man einen Tensor zu einem Vektor umformt

4.4.10 Eigenschaften von CNNs

Parameter/Weight-Sharing
ERkennung von gewissen Features (Ecken und Kanten) Bewegungs-Invarianz (durch Translations-Invarianz), Helligkeit-Invarianz alles folgend von geteilten Gewichten
Somit reduziert man die Information aufs Wesentliche und vernachlaessigt die Noise. Geteilte Gewichte
Lokale Konnektivitaet
Weil nicht fully-connected -> Overfitting erschwaert / unmoeglich

4.5 Autoencoder

4.5.1 Autoencoder

Lernen effiziente Datenrepraesenationen. Komprimierung. Unsupervised. Dimensionalitaetsreduktion.
Mathematik:

$$\phi : \mathcal{X} \to \mathcal{F} \tag{16}$$
$$\psi : \mathcal{F} \to \mathcal{X} \tag{17}$$
$$\phi, \psi = \arg\min_{\phi,\psi} \parallel X - (\phi \circ \psi)X \parallel^2 \tag{18}$$

4.5.2 Bedeutung von Autoencodern

4.5.2.1 Datenkompression

Falls ein Autoencoder nur aus linearen Neuronen besteht, ist die Dimensionsreduktion sehr aehnlich zu der Hauptkomponentenanalyse.
Problem: Netzwerk versucht Durchschnitt aller Trainingsdaten zu erlernen.

4.5.2.2 Datengenerierung

4.5.2.3 Eigenschaften erkennen

4.5.3 Variational Autoencoder?

4.5.4 Detangled Autoencoder?

4.6 Programmiertools

4.6.1 Tensorflow

Tensor

Ein Tensor ist eine verallgemeinerung von Matrizen auf n-dimensionale geometrische Objekte. Vorstellen kann man sich einen Tensor als einen Hyperrechteck mit n Dimensionen, innerhalb dessen die Zahlen in einem Raster angeordnet sind. Diese Zahlen sind die Elemente des Tensors. Dabei bezeichnet n den sogennaten Rang oder die Stufe des Tensors. Ein Tensor nullten Ranges ist ein Skalar, einer erster Stufe ein Vektor und Rang 2 ist eine Matrix. Tensoren haben noch viel weitreichendere Bedeutungen, vorallem fuer multilineare Abbildungen. Diese sind aber hier nicht relevant.

Ein Tensor dritten Ranges von der Form $(3 \times 3 \times 3)$ kann man sich folgendermassen vorstellen:

$$\overline{\mathbf{T}} \nexists [$$

5 Entwicklung eines Autoencoders

5.1 Tools

5.1.1 Python

5.1.2 Tensorflow

5.1.3 Keras

5.2 Idee

5.3 Voraussetzungen

5.3.1 Trainingsdaten

MegaFace Dataset

5.4 Programmieren

6 Diskussion?

7 Schlusswort

Abbildungsverzeichnis

1 das beschriebene Machine Learning Modells 6
2 beschriebenes Beispielmodells 7
3 Visualisierung des Gradientenabstieg 9
4 Auswirkungen verschiedener Lernraten 10
5 Perzeptron mit drei Inputs 11
6 erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts) . . . 12
7 Definition und Graph der Heaviside-Funktion Θ 13
8 Formel und Graph einer stueckweisen linearen Funktion 13
9 Definition, Ableitung und Graph der Sigmoid-Funktion σ 14
10 Schichten eines KNNs 14
11 zum Verstaentniss der Nomenklatur 15
12 Beispiel einer Bildmatrix 20
13 2-dimensionalen Schicht von Neuronen 20
14 eine Filtermatrix 20
15 Sobel-Filter angewandt auf Beispielsbild 20
16 Schema, wie ein Filter ueber ein Bild laueft 21
17 Schema eines CNNs 23

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth